Lab1Report

April 2, 2026

1 Lab 1 Report:

1.1 Data Preparation Techniques for Machine Learning

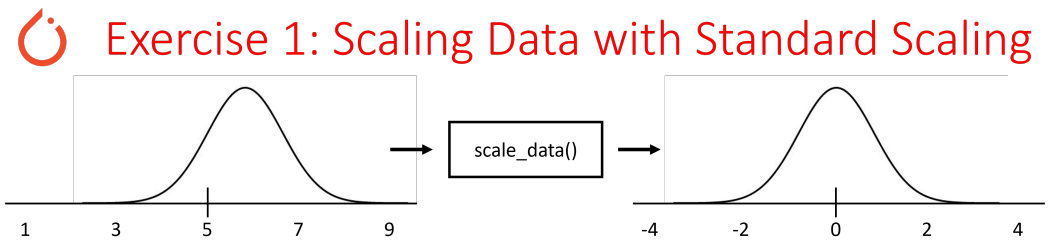
1.1.1 Name:

[558]:
```
#n Import necessary libraries

%matplotlib inline
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

[559]:
```
from IPython.display import Image # For displaying images in colab jupyter cell
```

[560]:
```
Image('../Lab_1_Template/lab1_exercise1.PNG', width = 1000)
```

[560]:



[561]:
```
# Load the dataset (.csv) using pandas package

CMS_calori_dataset = pd.read_csv('../Lab_1_Template/hgcal.csv')

# .head directive on the panda dataframe displays the first n-rows

CMS_calori_dataset.head(n = 10)
```

[561]:

	Unnamed: 0	x	y	z	eta	phi	energy	trackId
0	0	179.50383	-23.632137	-7.878280	-0.0435	-0.130900	0.200126	462412
1	1	-143.63881	110.217940	-72.706795	-0.3915	2.487094	2.734594	493395
2	2	179.50383	-23.632120	-146.429610	-0.7395	-0.130900	0.423910	1
3	3	-172.67310	54.443620	-238.065340	-1.0875	2.836160	0.713950	493640
4	4	-180.88046	7.897389	-238.065340	-1.0875	3.097959	0.000000	495225
5	5	-180.88045	-7.897438	-238.065340	-1.0875	-3.097959	0.034491	495225
6	6	-152.69838	-97.279590	-265.020540	-1.1745	-2.574361	0.580138	460126
7	7	-23.63213	179.503810	-325.172060	-1.3485	1.701696	0.411487	465028
8	8	-152.69835	97.279594	89.977780	0.4785	2.574361	0.183141	1383
9	9	-176.76110	39.187016	107.930240	0.5655	2.923426	0.337551	4421

[562]:
```
# Convert the panda dataframe into numpy 2D array

CMS_calori_dataset_np = CMS_calori_dataset.to_numpy()

# The converted numpy array has the dimension of 420 (rows) x 8 (columns)

print(CMS_calori_dataset_np.shape)
```

```
(420, 8)
```

[563]:
```
# Extract only x, y, z, eta, phi and energy columns from the dataset and stack␣
 ↪them along column direction
# Name this new 2D array CMS_calori_dataset_np_sub.
# The array should have dimension 420 (rows) x 6 (columns)
print(CMS_calori_dataset_np.shape)
CMS_calori_dataset_np_sub =CMS_calori_dataset_np[:, 1:7]
```

```
print(CMS_calori_dataset_np_sub.shape)
# YOUR CODE HERE
```

```
(420, 8)
(420, 6)
```

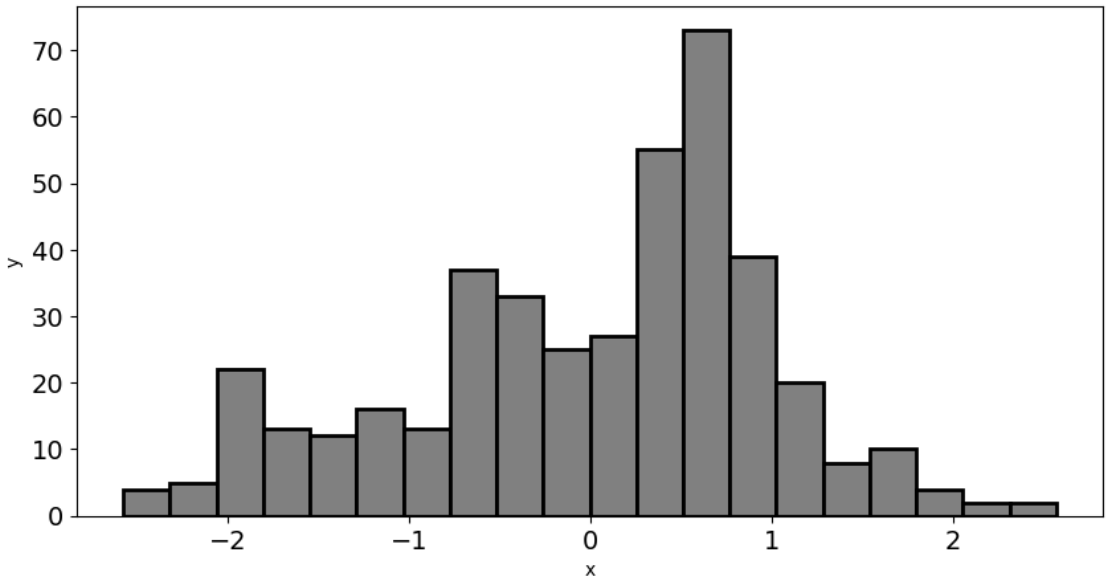
[564]:
```
# Create the scaling function

def scale_data(array_2d):
    # YOUR CODE HERE
    print("array:",array_2d, "end")
    mean = np.mean(array_2d, axis = 0);
    stddev = np.std(array_2d, axis = 0);
    out = np.divide(np.subtract(  array_2d,mean), stddev)
    return out
```

[565]:
```
# Test the function with CMS_calori_dataset_np_sub

CMS_calori_dataset_np_sub_scaled = scale_data(CMS_calori_dataset_np_sub)
print(CMS_calori_dataset_np_sub_scaled.shape)
```

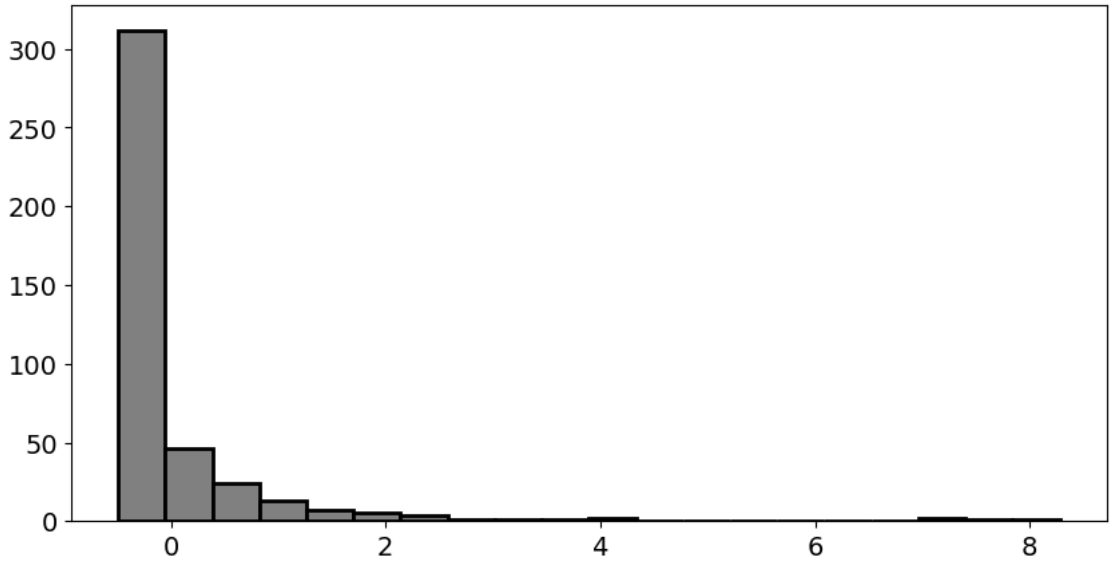
```
array: [[ 1.7950383e+02 -2.3632137e+01 -7.8782797e+00 -4.3500002e-02
  -1.3089974e-01  2.0012644e-01]
 [-1.4363881e+02  1.1021794e+02 -7.2706795e+01 -3.9149997e-01
   2.4870942e+00  2.7345939e+00]
 [ 1.7950383e+02 -2.3632120e+01 -1.4642961e+02 -7.3950000e-01
  -1.3089965e-01  4.2390990e-01]
 …
 [-3.5694687e+01 -5.0977300e+01  5.2279800e+02  2.8250000e+00
  -2.1816616e+00  5.3157926e-01]
 [-1.6106770e+01 -6.0111294e+01  5.2279800e+02  2.8250000e+00
  -1.8325956e+00  1.4225300e-01]
 [ 6.0111294e+01 -1.6106758e+01  5.2279800e+02  2.8250000e+00
  -2.6179916e-01  2.0927284e+00]] end
(420, 6)
```

[566]:
```
# Confirm the data is scaled for 'x' column

plt.figure(figsize = (10, 5))

plt.hist(CMS_calori_dataset_np_sub_scaled[:, 0], bins = 20, facecolor = 'grey',␣
 ↪edgecolor = 'black', linewidth = 2)
plt.xticks(fontsize=14)
plt.yticks(fontsize=14)

# Add proper x-label and y-label
plt.xlabel("x")
plt.ylabel("y")
```

```
# YOUR CODE HERE
plt.show()
```



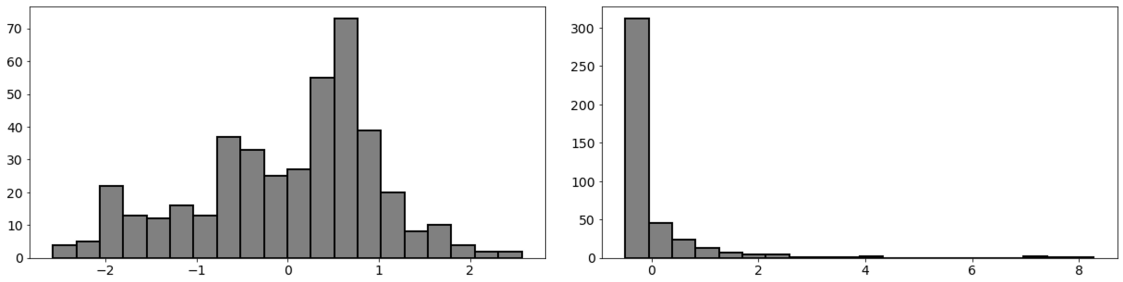
[567]:
```
# Confirm the data is scaled for 'energy' column

plt.figure(figsize = (10, 5))

plt.hist(CMS_calori_dataset_np_sub_scaled[:, 5], bins = 20, facecolor = 'grey',␣
 ↪edgecolor = 'black', linewidth = 2)
plt.xticks(fontsize=14)
plt.yticks(fontsize=14)

# Add proper x-label and y-label

# YOUR CODE HERE

plt.show()
```



1.1.2 Expected histogram outputs - Feel free to style your plot differently

[568]:
```
Image('../LAB_1_Template/lab1_e1_expected_outputs.PNG', width = 1000)
```

[568]:

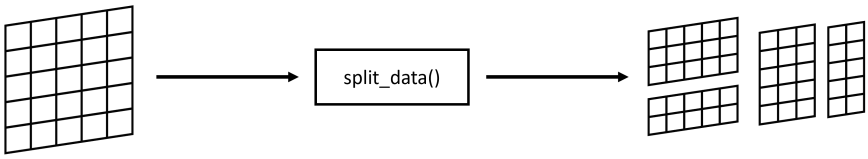


Scaled ‘x’ Scaled ‘energy’

[569]:
```
Image('../LAB_1_Template/lab1_exercise2.PNG', width = 1000)
```

[569]:

Exercise 2: Data Splitting



- In this exercise you will write a function called split_data() which given a NumPy array, it splits the array into sub-arrays.
- Data splitting is used to divide the dataset into training, validation and testing sets, which we will describe in later lab.
- The function should take following parameters
  - arr – 2D NumPy array representing a dataset
  - split_proportions – a list containing split ratios, e.g., [0.2, 0.3, 0.5]
  - axis – a direction to be splitted (0 = row-wise, 1 = column-wise)
- Test your function on the scaled dataset from exercise 1 with given parameters in the lab template.
- Confirm that your sub arrays have correct dimensions by printing their shape

68

[570]:
```
# Create the splitting function

def split_data(arr:np.ndarray, split_proportions:list[float], axis):

    # YOUR CODE HERE
    # Returns a list of numpy sub-arrays according to split proportions
    dims = arr.shape;
    split_data_list = [];
    count = dims[axis];
    indx = 0
    indices = []
    for i in split_proportions:
        indx = indx+count*i;
        indices.append(int(indx));
    split_data_list = np.split(arr,indices, axis)
    return split_data_list
```

[571]:
```
# Test your split function against scaled CMS Calorimieter dataset from␣
 ↪exercise 1

sub_data_list_1 = split_data(arr = CMS_calori_dataset_np_sub_scaled,
                                    split_proportions = [0.6, 0.2, 0.2], axis =␣
 ↪0)
```

[572]:
```
# Confirm that dataset has been split into correct shapes
# The correct dimensions should be (252, 6) (84, 6) (84, 6)
```

```
print(sub_data_list_1[0].shape, sub_data_list_1[1].shape, sub_data_list_1[2].
 ↪shape)
```

```
(252, 6) (84, 6) (84, 6)
```

[573]:
```
# Test your split function against scaled CMS Calorimieter dataset from␣
 ↪exercise 1

sub_data_list_2 = split_data(arr = CMS_calori_dataset_np_sub_scaled,
                                                split_proportions = [0.5, 0.5],␣
 ↪axis = 1)
```

[574]:
```
# Confirm that dataset has been split into correct shapes
# The correct dimensions should be (420, 3) (420, 3)

print(sub_data_list_2[0].shape, sub_data_list_2[1].shape)
```

```
(420, 3) (420, 3)
```